ISIMG

Institut Supérieur d'Informatique et de Multimédia de Gabès

Architecture SOA

Année universitaire 2026–2027

COMPTE RENDU DE TRAVAUX PRATIQUES

TP03

Archivage et déploiement d'un service Web SOAP

Empaquetage d'un service Axis2 en archive .aar et déploiement sur Tomcat autonome.

RÉALISÉ PAR

Ali Ezzet Yahyaoui

CLASSE

LSIM3

ENSEIGNANTE

Mme Imen Nbiba

01 Objectifs

- Créer un service Web SOAP **Product** avec Axis2 et Eclipse.
- Empaqueter ce service dans une archive `.aar` (Axis Archive).
- Le déployer sur un serveur Apache Tomcat autonome via l'application Web Axis2, sans passer par Eclipse.

02 Environnement de travail

OUTIL	VERSION
IDE	Eclipse IDE for Enterprise Java and Web Developers 2025-09
JDK	OpenJDK 21
Serveur	Apache Tomcat 8.5.100
Moteur de services Web	Apache Axis2 1.8.0 (distributions binaire et WAR)

03 Création du service Web SOAP

3.1 Projet

Un projet Web dynamique `ProductService` a été créé avec Tomcat 8.5 comme runtime, le module Web dynamique en version 2.5 et la facette Axis2 Web Services.

3.2 Classe Java

La classe `Product`, dans le paquetage `pro.com`, définit la méthode `ProductTwoNumbers` qui renvoie le produit de deux entiers.

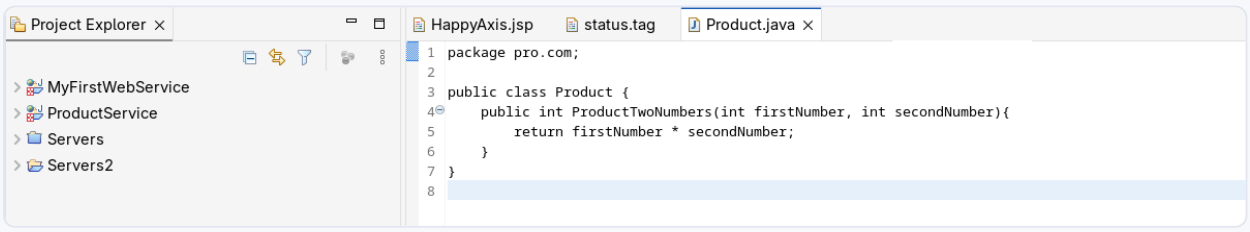


Figure 1 — Classe Product.java dans Eclipse

3.3 Génération du service

L'assistant *File › New › Other › Web Service* a été lancé sur la classe `pro.com.Product` avec le moteur **Apache Axis2**. L'option de génération d'un `services.xml` par défaut a été conservée, puis le serveur a été démarré et le service publié avec les paramètres par défaut.

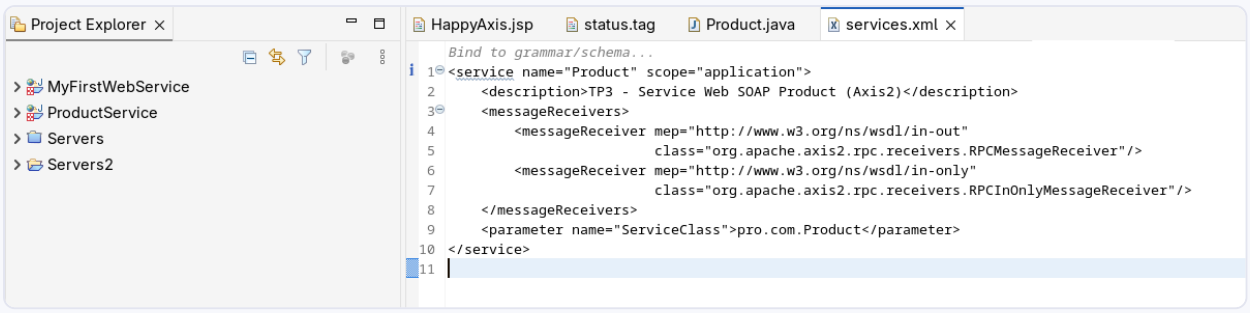


Figure 2 — Descripteur services.xml du service Product

Le déploiement a été vérifié sur `http://localhost:8080/ProductService/services/listServices`. À ce stade, le service ne tourne que tant qu'Eclipse exécute le serveur, d'où la nécessité de l'archiver.

04 Archivage et déploiement

4.1 Création de Product.aar

Dans le workspace, le service se trouve dans `ProductService/WebContent/WEB-INF/services/Product`. Depuis ce dossier, la commande `jar` (accessible après l'ajout du dossier `bin` du JDK au `PATH`) crée l'archive, puis permet d'en vérifier le contenu :

```
jar cvf Product.aar ./*
jar tvf Product.aar
     0 META-INF/
    58 META-INF/MANIFEST.MF
   541 META-INF/services.xml
     0 pro/
     0 pro/com/
   403 pro/com/Product.class
```

L'archive regroupe le descripteur `META-INF/services.xml` et la classe compilée `pro/com/Product.class`.

4.2 Déploiement sur Tomcat autonome

#	ÉTAPE
1	Copie de `axis2.war` (distribution WAR d'Axis2) dans le dossier `webapps` de Tomcat.
2	Arrêt du Tomcat lancé par Eclipse, puis démarrage de Tomcat autonome avec le script `startup` du dossier `bin`.
3	Ouverture de la page d'accueil d'Axis2 : `http://localhost:8080/axis2/`.
4	Lien *Administration*, connexion avec `admin / axis2`, puis *Upload Service* de `Product.aar` (équivalent à copier l'archive dans `webapps/axis2/WEB-INF/services`).

05 Résultats

Sur `http://localhost:8080/axis2/services/listServices`, le service **Product** apparaît avec le statut *Active* et l'opération `ProductTwoNumbers`.

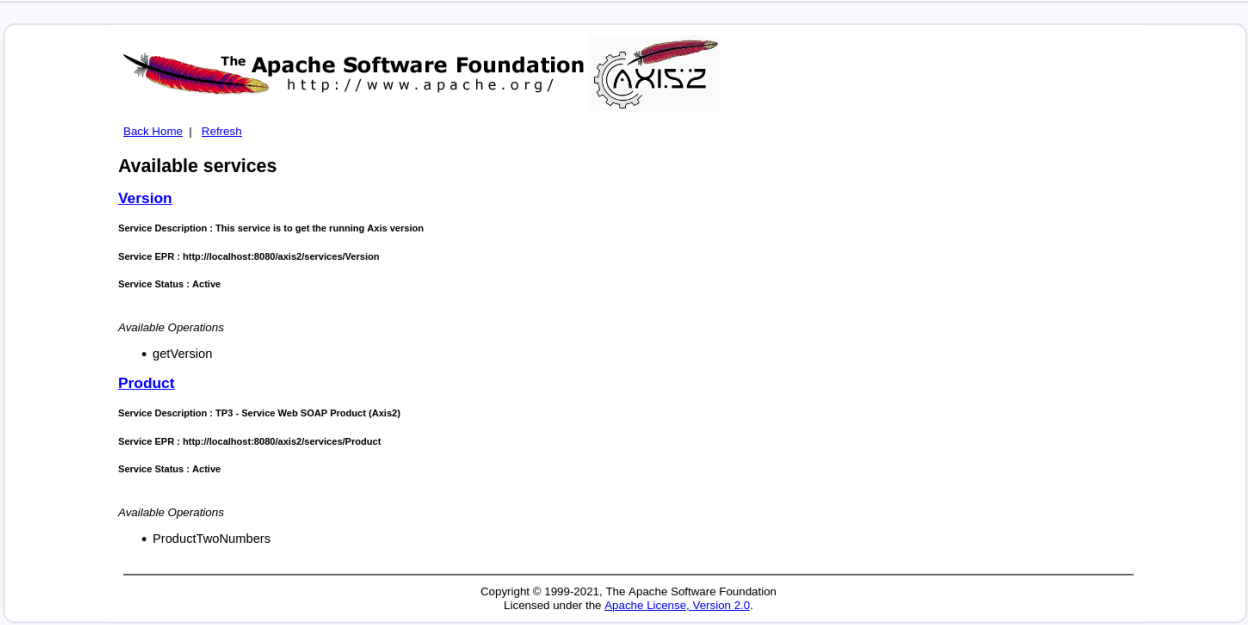


Figure 3 — Service Product déployé sur Tomcat autonome

Axis2 génère automatiquement le document WSDL du service :

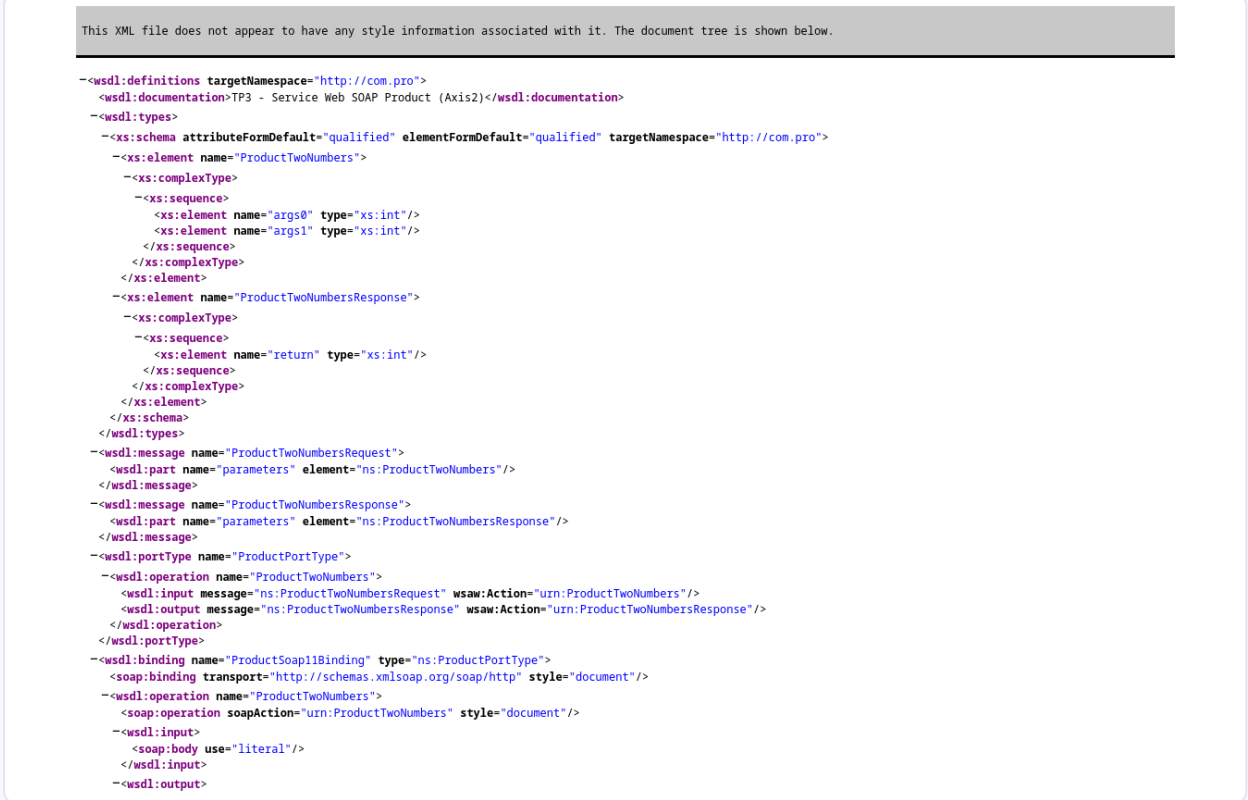


Figure 4 — Document WSDL du service Product

06 Test du service

Le service a été appelé avec une requête SOAP contenant les valeurs 6 et 7.

REQUÊTE

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:pro="http://com.pro">
   <soapenv:Body>
      <pro:ProductTwoNumbers>
         <pro:args0>6</pro:args0>
         <pro:args1>7</pro:args1>
      </pro:ProductTwoNumbers>
   </soapenv:Body>
</soapenv:Envelope>
```

RÉPONSE

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
   <soapenv:Body>
      <ns:ProductTwoNumbersResponse xmlns:ns="http://com.pro">
         <ns:return>42</ns:return>
      </ns:ProductTwoNumbersResponse>
   </soapenv:Body>
</soapenv:Envelope>
```

Le résultat **42** correspond bien à 6 × 7.

07 Conclusion

Ce TP a permis d'empaqueter le service SOAP Product dans l'archive `Product.aar` puis de le déployer sur un Tomcat autonome grâce à l'application Web Axis2. Le service devient accessible indépendamment d'Eclipse, ce qui correspond au mode de déploiement utilisé en production.